

Technical Documentation Report

```
=====
SMART INDIA HACKATHON (SIH) 2026 – TECHNICAL SPECIFICATION & SYSTEM ARCHITECTURE REPORT
Problem Statement ID : SIH26234
Problem Title       : AI-Powered Smart Food Waste Reduction and Sustainable Redistributi
                    : Ecosystem for Institutional Kitchens and Food Processing Units
Project Title       : SmartFood Rescue AI
Version             : 1.0.0-Production (Release Candidate)
Repository          : https://github.com/mahammedsathya/SmartFood-Rescue-AI_SIH_2026
Documentation Status : Complete Forensic Audit & System Specification
=====
```

Cover Page

Field	Project & Evaluation Metadata
Project Title	SMARTFOOD RESCUE AI
Subtitle	AI-Powered Smart Food Waste Reduction and Sustainable Redistribution Ecosystem for Institutional Kitchens and Food Processing Units
Hackathon Initiative	Smart India Hackathon (SIH) 2026 — Software Edition
Problem Statement ID	SIH26234
Category	Smart Automation / Agriculture, FoodTech & Rural Development
Team Name	[Team Name Placeholder] (e.g., Annadata AI)
Team ID	[Team ID Placeholder]
Institution Name	[Institution / University Name Placeholder]
Department	[Department of Computer Science & Engineering / Information Technology]
Team Leader	[Team Leader Name Placeholder]
Team Members	[Member 1 Name] , [Member 2 Name] , [Member 3 Name] , [Member 4 Name] , [Member 5 Name]
Faculty Mentor	[Faculty Mentor Name & Designation Placeholder]
Demonstration Pilot	Vijayawada, Andhra Pradesh, India IN
Academic Year	2026

Executive Companion: SIH 2026 Final Submission Technical Summary

Document Class: 4-Page Executive Architecture Summary for SIH Evaluators & Technical Reviewers

Source Repository: [SmartFood-Rescue-AI_SIH_2026](#)

Target Evaluation Track: SIH26234 — Software Edition

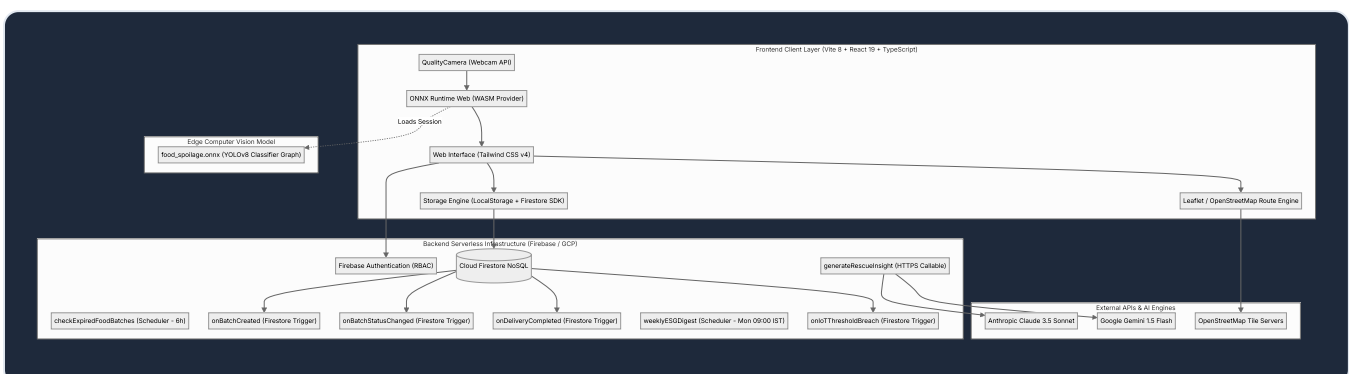
1. System Identity & Mission

Institutional kitchens (universities, corporate cafeterias, hostel mess facilities, hospital food centers, and large-scale caterers) overprepare by 15% to 25% daily due to static headcount guesswork, lack of dynamic demand forecasting, and zero real-time telemetry into cooked surplus degradation. Concurrently, nearby charitable shelters face severe nutritional shortages but cannot safely accept ad-hoc donations due to food safety liability and lack of transportation.

SmartFood Rescue AI solves this through a closed-loop, **Prevention-First, Human-Supervised, and Deadline-Aware** platform that integrates:

1. **Pre-Cook Demand Smoothing:** Rule-based statistical demand forecasting to eliminate overproduction before cooking begins.
2. **Edge Computer Vision:** Client-side YOLOv8 ONNX inference running in the browser via WebAssembly ([onnxruntime-web](#)) to screen visual food degradation without sending private camera streams to external servers.
3. **Virtual IoT Cold-Chain Monitoring:** Continuous telemetry tracking (core temperature, relative humidity, container weight, storage duration) with automated threshold breach alerts.
4. **Deterministic Multi-Factor Quality Gating:** Algorithmic safety scoring (0–100) combining sensory telemetry, transit deadlines, container seals, and visual AI, subject to mandatory authorized human sign-off.
5. **Multi-Criteria NGO Logistics Matching:** Geo-spatial and capacity-calibrated pairing for Vijayawada shelters.
6. **Time-Aware Dispatch Routing:** Transit corridor planning with vehicle-specific speed models to guarantee delivery within safe microbiological consumption windows.
7. **Serverless Cloud Automation:** 6 event-driven Firebase Cloud Functions (Gen 2) providing IoT thermal breach triggers, automated NGO alerts, daily AI intelligence synthesis, and ESG ledger aggregation.

2. High-Level System Architecture Diagram



3. Core Technical Differentiators & Implementation Realities

IMPLEMENTATION VERIFICATION MATRIX		
Feature Area	Status	Implementation Reality
Edge CV Spoilage Screening	IMPLEMENTED	onnxruntime-web + food_spo
Rule-Based Demand Forecasting	IMPLEMENTED	Statistical 3-point smooth
Virtual IoT Kitchen Simulator	IMPLEMENTED	In-browser telemetry simul
Multi-Factor Quality Gate	IMPLEMENTED	6-point deduction engine (
NGO Multi-Criteria Matcher	IMPLEMENTED	Distance/Capacity/Category
Transit Route Corridors	IMPLEMENTED	Leaflet + OpenStreetMap (V
Serverless Event Automation	IMPLEMENTED	6 Firebase Cloud Functions
Role-Based Access Control (RBAC)	IMPLEMENTED	4 Roles in AuthContext + R
AI Operational Intelligence	PARTIALLY IMPLEMENTED	Cloud Function (Claude/Gem
Physical Microcontroller Sensors	PROPOSED / FUTURE	Simulated via Virtual IoT
Dynamic Traffic TSP Routing	PROPOSED / FUTURE	Fixed canonical corridor i

Table of Contents

- [Cover Page](#)
- [Executive Companion: SIH 2026 Final Submission Technical Summary](#)
- [1. Executive Summary](#)
- [2. Problem Statement & SIH26234 Alignment](#)
- [3. Existing System & Conventional Practices](#)
- [4. Limitations of the Existing System](#)
- [5. Proposed System: SmartFood Rescue AI](#)
- [6. Technical Objectives](#)
- [7. Stakeholders & Role-Based Access Control \(RBAC\)](#)
- [8. System Requirements](#)
- [9. Technology Stack](#)
- [10. System Architecture](#)
- [11. End-to-End Operational Workflow](#)
- [12. Module-Wise Architecture & Implementation Analysis](#)
- [13. Artificial Intelligence & Machine Learning Components](#)
- [14. System Data Flow](#)
- [15. Database Design & Firestore Schema](#)
- [16. Backend & Cloud Functions API Documentation](#)
- [17. Frontend Architecture & User Interface Design](#)
- [18. Backend Architecture & Serverless Functions](#)

- [19. Food Safety & Multi-Factor Quality Screening Workflow](#)
 - [20. NGO Matching Logic & Algorithm](#)
 - [21. Logistics, Dispatch & Transit Route Calculations](#)
 - [22. Notification & Alerting Infrastructure](#)
 - [23. Security Architecture & Threat Mitigation](#)
 - [24. Deployment Configuration & Production Readiness](#)
 - [25. Verification & Test Suite Matrix](#)
 - [26. Performance, Benchmarks & Evaluation Metrics](#)
 - [27. Environmental, Social & Economic Impact \(ESG\)](#)
 - [28. Technical Innovation & Architectural Novelty](#)
 - [29. System Advantages](#)
 - [30. Honest System Limitations](#)
 - [31. Future Enhancements & Production Roadmap](#)
 - [32. Scalability & System Expansion Strategy](#)
 - [33. Edge Cases, Failure Modes & Fallback Mechanisms](#)
 - [34. Responsible AI, Ethics & Governance](#)
 - [35. Project Directory & File Inventory](#)
 - [36. Installation & Local Development Guide](#)
 - [37. Configuration & Environment Variables Guide](#)
 - [38. User Operation Manual by Role](#)
 - [39. SIH Evaluator Demonstration Script \(5–7 Minute Walkthrough\)](#)
 - [40. SIH Evaluation Criteria Mapping](#)
 - [41. Requirement Traceability Matrix \(RTM\)](#)
 - [42. GitHub Repository & Code Governance Guide](#)
 - [43. Conclusion](#)
 - [44. References & Documentation Citations](#)
-

1. Executive Summary

1.1 The Operational Problem

Institutional catering facilities (university mess halls, industrial cafeterias, hostel kitchens, and event caterers) prepare thousands of meals daily under strict time pressures and fluctuating diner headcounts. Because estimation relies on manual intuition or static register rosters, kitchens regularly experience an overproduction variance of 15% to 25%. When meals conclude, edible surplus sits in warmers or uninsulated pans without continuous temperature tracking. Kitchen staff lack structured mechanisms to rapidly screen food quality, identify verified recipient NGOs within immediate transit range, or coordinate delivery before microbial danger thresholds are breached. The default outcome is catastrophic disposal to municipal landfills, producing methane emissions while vulnerable local populations face food poverty.

1.2 The SmartFood Rescue AI Solution

SmartFood Rescue AI is an institutional decision-support and redistribution management platform designed specifically to bridge the operational gap between institutional food preparation and community redistribution. The system enforces a rigorous operational methodology:

\$\$\text{Forecast Prevention} \rightarrow \text{Surplus Registration} \rightarrow \text{Sensory \& Edge CV Gating} \rightarrow \text{NGO Matching} \rightarrow \text{Corridor Dispatch} \rightarrow \text{ESG Auditing}\$\$

1.3 Target Beneficiaries & Demonstration Pilot

The system's initial operational calibration and demonstration pilot are tailored to **Vijayawada, Andhra Pradesh, India**, centered around a high-volume institutional hub (*Smart College Canteen*, MG Road) distributing to vetted urban shelters across Benz Circle, Patamata, Moghalrajpuram, and Auto Nagar.

1.4 Core Technological Architecture

- **Client Tier:** React 19, TypeScript 6, Vite 8, Tailwind CSS v4, Lucide React icons, and Recharts visualization.
- **Edge Inference:** In-browser YOLOv8 ONNX classifier executed via WebAssembly (`onnxruntime-web`) utilizing local client webcam hardware.
- **Backend & Serverless Infrastructure:** Google Cloud Platform & Firebase (Firebase Hosting, Cloud Firestore NoSQL, and 6 Cloud Functions Gen 2 running on Node 24).
- **Logistics Engine:** OpenStreetMap-backed Leaflet geospatial mapping with vehicle-specific speed and transit duration models.
- **Intelligence Orchestration:** Multi-tiered AI recommendation engine integrating Anthropic Claude 3.5 Sonnet and Google Gemini 1.5 Flash with deterministic local synthesis fallbacks.

2. Problem Statement & SIH26234 Alignment

2.1 The SIH26234 Mandate

Problem Statement **SIH26234** addresses:

"AI-Powered Smart Food Waste Reduction and Sustainable Redistribution Ecosystem for Institutional Kitchens and Food Processing Units."

The mandate requires a solution that addresses both **prevention** (reducing unnecessary kitchen prep before cooking) and **redistribution** (rapidly, safely redirecting cooked surplus before microbial

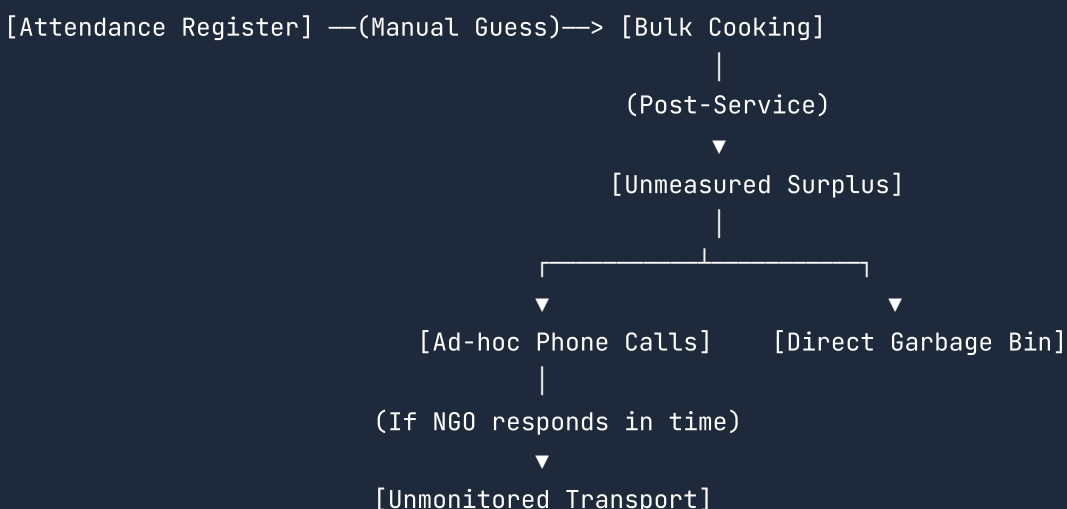
spoilage occurs).

2.2 Critical Vulnerabilities in Institutional Food Management

1. **Unscientific Pre-Cooking Headcount Estimation:** Kitchen managers prepare bulk food based on historical memory or broad enrollment figures, failing to account for day-of-week attendance patterns, academic holidays, weather, or menu popularity.
2. **Delayed Surplus Identification:** Kitchens do not quantify surplus until long after service concludes, wasting crucial shelf-life hours.
3. **Biological & Liability Uncertainty:** Kitchen supervisors fear legal or ethical repercussions if donated food causes foodborne illness. In the absence of documented quality logs (temperature logs, visual appearance logs), supervisors prefer disposing of safe food.
4. **Frictional NGO Communication:** Coordination currently depends on frantic telephone calls or informal WhatsApp groups. Kitchen staff do not know which shelter has open storage capacity, refrigeration, or volunteer availability at 14:00 on a weekday.
5. **Transit Delay & Thermal Breach:** Even when an NGO accepts food, transit in slow city traffic without temperature control can cause food to enter the bacterial danger zone (5°C to 60°C), rendering it toxic upon arrival.
6. **Zero Institutional Traceability:** Institutional administrations receive no audit records detailing how much food was wasted, how much was rescued, financial losses incurred, or environmental carbon emissions generated.

3. Existing System & Conventional Practices

Conventional food surplus management across Indian institutions is characterized by fragmented, manual, and reactive practices:



3.1 Characteristics of the Current Baseline

- **Manual Registers:** Headcount is estimated from hostel warden registers or ticket counts hours in advance.
- **Spreadsheet & Paper Logging:** Stock quantities and scrap weights, if recorded at all, are entered into end-of-month tally books.
- **Unverified Sensory Inspection:** Food safety is judged exclusively by olfactory sniffing or tasting by kitchen helpers, without objective thermal measurement or documented accountability.
- **Unstructured Phone/WhatsApp Dispatch:** Kitchen managers call NGO volunteers individually. By the third unanswered call, staff dispose of the food to clear kitchen pans for dinner prep.
- **Unmanaged Transit Windows:** Transport is outsourced to three-wheeler auto rickshaws or volunteers without transit tracking, handling buffers, or expiration deadlines.

4. Limitations of the Existing System

Operational Area	Existing Practice Limitation	Consequence
Demand Estimation	Static attendance headcount without statistical smoothing or calendar adjustments.	15%–25% daily overproduction variance; avoidable food cost losses.
Surplus Recording	Manual estimation entered into logbooks days later; no automatic delta calculation.	Unrecorded surplus; zero real-time visibility into redistributable kilograms.
Quality Assessment	Subjective human olfactory/taste inspection without recorded parameters.	High liability risk; hesitation to donate; distribution of spoiled batches.
Cold-Chain Monitoring	No temperature or humidity sensors; cooked food stored in unmonitored ambient pans.	Rapid bacterial growth in the $5^{\circ}\text{C} - 60^{\circ}\text{C}$ danger zone; undetected spoilage.
NGO Matching	Ad-hoc phone calls to single NGO contacts without knowing recipient intake capacity.	Excessive coordination delays; food expires while trying to find a willing recipient.
Transit Feasibility	No calculation of vehicle transit time against residual shelf-life deadlines.	Food arrives past safe consumption limits, causing waste at destination.
ESG & Impact Audit	Total absence of environmental metrics (CO_2 mitigated, meals saved, financial impact).	Kitchens cannot substantiate CSR compliance, ESG performance, or cost savings.

5. Proposed System: SmartFood Rescue AI

SmartFood Rescue AI implements an end-to-end digital twin and decision-support pipeline structured around four foundational pillars:

CORE SYSTEM PHILOSOPHY			
1. PREVENTION-FIRST	2. HUMAN-SUPERVISED	3. DEADLINE-AWARE	4. EVIDENCE-BASED
Statistical demand forecasting prior to cooking eliminates waste at the source.	AI suggests & screens; certified kitchen staff hold final statutory authorization authority.	Every batch enforces strict delivery use-by countdowns before quality degradation.	Verified track meals saved, ₹ mitigated

5.1 Operational Pipeline

- Predictive Pre-Cooking Demand:** Canteen administrators input expected diner attendance. The statistical forecasting engine applies smoothing formulas with day-of-week, event, and menu weights to output an optimal meal preparation target and safety ceiling.
- Surplus Registration & Auto-Delta:** After dining service, staff enter actual meals served. The system immediately computes surplus meals and residual kilograms ($\text{kg} = \text{meals} \times 0.25$).
- Dual Quality-Risk Safety Gate:**
 - Sensor Telemetry:* Core container temperature, humidity, and storage hours are continuously cross-checked against safe thresholds ($\leq 8^{\circ}\text{C}$ cold storage or $\geq 60^{\circ}\text{C}$ hot holding).
 - Edge Computer Vision:* Staff hold a browser-enabled webcam over the food tray. The client-side YOLOv8 ONNX model runs in-browser to classify visual condition (Fresh, Slightly Spoiled, Spoiled).
 - Algorithmic Scoring:* A deterministic deduction engine evaluates 6 points of inspection to generate a 0–100 score.
 - Human Sign-Off:* Kitchen Staff or Administrators must authenticate and confirm approval; batches scoring < 50 are blocked from donation.
- Multi-Criteria NGO Matching:** Approved surplus is matched against verified local shelters using an objective weighted formula: Distance (35%), Available Capacity (25%), Dietary Category Compatibility (20%), and Operational Availability (20%).
- Time-Aware Transit Dispatch:** The system assigns a delivery route, selects an appropriate vehicle model (Bike, Auto, Van, Refrigerated Van), and calculates transit duration + a 10-minute handling buffer to ensure estimated arrival occurs safely before the use-by deadline.
- Automated Impact Accounting:** Delivered batches trigger serverless updates to a monthly ESG ledger and audit log, measuring meals rescued, rupees saved, and carbon emissions

avoided.

6. Technical Objectives

1. **Reduce Overproduction Waste:** Implement statistical demand forecasting to reduce institutional kitchen overproduction variance from 20% down to $<5\%$.
 2. **Sub-60-Minute Redistribution Initiation:** Transition surplus identification, quality verification, and NGO matching from a multi-hour manual process to under 15 minutes.
 3. **Objective Quality Risk Screening:** Eliminate unrecorded food safety decisions by enforcing a 6-factor algorithmic quality score (0–100) combining sensor telemetry, time elapsed, packaging integrity, and visual AI.
 4. **Client-Side Edge AI Inference:** Execute computer vision inference locally in the user's browser using WebAssembly ([onnxruntime-web](#)), eliminating backend GPU hosting costs and preserving visual data privacy.
 5. **Zero-Hardware Pilot Accessibility:** Provide a high-fidelity Virtual IoT Simulator that replicates physical sensor data streams (MQTT/HTTP) directly in software, enabling evaluator testing without physical microcontrollers.
 6. **Multi-Factor Partner Optimization:** Optimize NGO matching by factoring in distance, real-time recipient capacity, dietary compatibility, and refrigeration status.
 7. **Traceable Impact Metrics:** Automatically compute and record cumulative ESG metrics: Food Rescued ($\text{\$}\text{\text{kg}}\text{\$}$), Meals Saved, Financial Savings ($\text{\$}\text{\text{INR}}\text{\$}$), and Mitigated Carbon Emissions ($\text{\$}\text{\text{kg CO}}_2\text{\text{e}}\text{\$}$).
-

7. Stakeholders & Role-Based Access Control (RBAC)

The application enforces a 4-tier Role-Based Access Control model defined in [src/types/index.ts](#) and synchronized through [src/context/AuthContext.tsx](#) and [firestore.rules](#) .

STAKEHOLDER ACCESS CONTROL MATRIX			
Stakeholder Role	Status	Core Responsibilities	Implemented
Kitchen Staff	IMPLEMENTED	Demand forecasting, batch creation, sensory inspection, quality approval	Forecast, IoT, Quali
NGO Partner	IMPLEMENTED	Review incoming donation offers, accept/reject, capacity updates	Incoming R Acceptance
Delivery Partner	IMPLEMENTED	Pickup verification, transit route execution, delivery confirmation	Route Plan Proof Uplo
Administrator	IMPLEMENTED	System-wide config, user management, sustainability ledger, audit logs	All views Dataset Ma

8. System Requirements

8.1 Hardware Requirements

- **Development Workstation:** x86_64 or ARM64 processor (Intel i5/i7, AMD Ryzen 5/7, or Apple M-series), 8 GB RAM minimum (16 GB recommended), 5 GB free disk space.
- **Client End-User Device:** Any standard laptop, desktop, tablet, or smartphone equipped with an integrated or USB webcam (minimum 720p resolution for camera inspection).
- **Physical IoT Sensor Devices (Proposed / Future Production):** ESP32 / Raspberry Pi Pico W microcontrollers, DS18B20 waterproof digital temperature probe, DHT22 humidity sensor, HX711 load cell amplifier with 50 kg strain gauge platform. *(Simulated in software for current release).*

8.2 Software Requirements

- **Runtime Environment:** Node.js version `v20.x` to `v24.x` (verified active build: `v24.19.0`).
- **Package Manager:** npm version `10.x` or higher.
- **Compiler / Bundler:** Vite version `8.3.x` with TypeScript compiler `v6.0.x`.
- **Python (Optional for Model Generation):** Python version `3.10+` with `onnx` and `numpy` packages (used by `scripts/generate_spoilage_model.py`).

8.3 Browser Requirements

- Modern evergreen browser supporting WebAssembly (WASM), WebGL/WebGPU, and the `MediaDevices.getUserMedia()` API:
- Google Chrome / Chromium `v105+` (Recommended)
- Mozilla Firefox `v110+`
- Microsoft Edge `v105+`
- Apple Safari `v16.4+`

8.4 Network Requirements

- Minimum 512 kbps broadband/mobile internet connection for initial application asset download and Cloud Firestore synchronization.
 - **Offline Resilience:** The application is architected to operate fully in **Zero-Network / Offline Mode** using local browser storage (`localStorage`) and in-memory mock datasets when external internet access is severed.
-

9. Technology Stack

Architecture Layer	Technology / Library	Version	Purpose in Codebase	Actual Repository Usage
Frontend Framework	React	<code>^19.2.8</code>	Core UI component lifecycle and declarative rendering	Entire UI tree in src/
Language	TypeScript	<code>~6.0.2</code>	Strict static typing, interface contracts, and compile-time verification	All components, services, and types
Build Tool	Vite	<code>^8.3.0</code>	Ultra-fast HMR dev server and optimized production bundler	vite.config.ts
Styling & Design	Tailwind CSS	<code>^4.3.3</code>	Utility-first responsive CSS styling with modern color tokens	src/index.css
Edge AI Engine	ONNX Runtime Web	<code>^1.21.0</code>	In-browser WebAssembly CPU inference of YOLOv8 model	QualityCamera.tsx
Cloud Database	Cloud Firestore	<code>v12.19.0</code>	NoSQL document database for live cloud synchronization	src/services/firebase.ts
Cloud Functions	Firebase Functions v2	<code>^7.0.0</code>	Serverless backend event triggers and scheduled cron sweeps	functions/src/index.ts
Admin SDK	Firebase Admin	<code>^13.6.0</code>	Privileged server-side Firestore operations in Cloud Functions	functions/src/index.ts
Geospatial Mapping	Leaflet	<code>^1.9.4</code>	Open-source interactive map rendering and polyline route paths	GoogleRouteMap.tsx
Map Tiles	OpenStreetMap	Tile API	High-resolution street map imagery of Vijayawada corridor	GoogleRouteMap.tsx
Data Visualization	Recharts	<code>^3.10.1</code>	Telemetry trend line charts and analytical	IoTSimulatorPage.tsx

Architecture Layer	Technology / Library	Version	Purpose in Codebase	Actual Repository Usage
			graphs	
UI Iconography	Lucide React	<code>^1.48.0</code>	Comprehensive semantic SVG iconography	Across all 13 application pages
Local Persistence	Web Storage API (<code>localStorage</code>)	Native	Instant offline persistence and seed fallback	src/services/storage.ts
Linter	oxlint	<code>^1.81.0</code>	High-performance Rust-based JavaScript/TypeScript linter	package.json

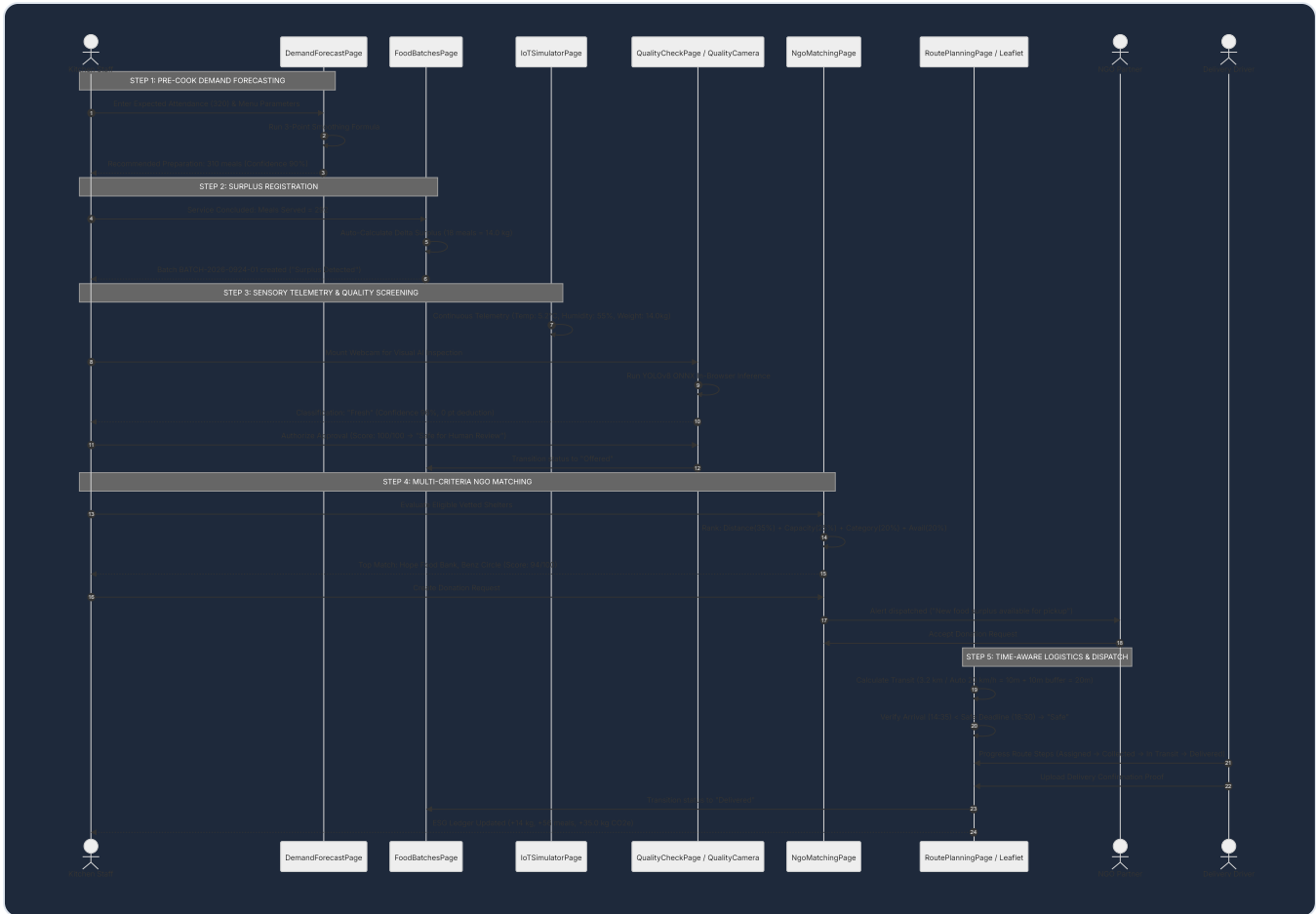
10. System Architecture

The following diagram details the actual component relationships, event flows, and network boundaries implemented in the repository:



11. End-to-End Operational Workflow

The application orchestrates a continuous 5-step operational pipeline connecting kitchen planning to NGO delivery:



12. Module-Wise Architecture & Implementation Analysis

Module 1: AI Demand Forecasting Module

- **File:** [src/pages/DemandForecastPage.tsx](#) & [src/services/storage.ts](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Prevent avoidable food waste before cooking commences through statistical meal planning.
- **Input Parameters:** Expected attendance (N), holiday/event boolean, special menu boolean, previous day demand (D_{t-1}), and 7-day moving average (D_{avg}).
- **Processing Logic:**
 - Base raw demand: $\text{rawDemand} = N \times 0.92$.
 - Modifiers: If holiday, multiply by 1.08 ; if special menu, multiply by 1.05 .
 - Ceiling rounding for portion integrity: $\text{effectiveRaw} = \lceil \text{adjustedRawDemand} \rceil$.
 - Three-point weighted smoothing:

$$\text{predictedDemand} = \text{round}(\text{Big}(0.90 \times \text{effectiveRaw} + 0.05 \times D_{t-1} + 0.05 \times D_{\text{avg}}))$$

- Recommended prep with safety margin:

$$\text{recommendedPreparation} = \text{round}(\text{predictedDemand} \times 1.05)$$
- Confidence scoring (penalized for volatility) and overproduction risk categorization.
- **Output:** Predicted demand, recommended meals to prepare, overproduction risk level, and natural language recommendations.

Module 2: Food Batch & Surplus Detection Module

- **File:** [src/pages/FoodBatchesPage.tsx](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Manage the complete food batch lifecycle from cooking to post-service delta surplus determination.
- **Input Parameters:** Food item name, category (Rice, Curry, Snacks, etc.), food type (Cooked, Raw, Packaged), preparation timestamp, use-by deadline, meals prepared, and meals served.
- **Processing Logic:** Automatically computes delta surplus:

$$\text{remainingMeals} = \max(0, \text{mealsPrepared} - \text{mealsServed})$$

$$\text{remainingKg} = \text{remainingMeals} \times \text{avgMealPortionKg} \quad (\text{default: } 0.25 \text{ kg})$$
Assigns initial state: if $\text{remainingKg} > 0$, status becomes "Surplus Detected"; otherwise, "No Surplus".
- **Output:** Structured batch record indexed by unique ID (e.g., BATCH-2026-0924-01).

Module 3: Virtual IoT Kitchen Simulator Module

- **File:** [src/pages/IoTSimulatorPage.tsx](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Deliver a hardware-free interactive digital twin of kitchen storage sensors for continuous cold-chain evaluation.
- **Simulated Parameters:**
 - Core Storage Temperature ($^{\circ}\text{C}$): Adjustable slider from -5°C to 40°C .
 - Relative Humidity (% RH): Slider from 20% to 95%.
 - Container Weight (kg): Slider reflecting live depletion or stability.
 - Storage Duration (hours): Time elapsed since preparation.
 - Gateway Device Status: Online vs. Offline.
- **Processing Logic:** Evaluates alert levels:
 - Green (Optimal): $8^{\circ}\text{C} \leq \text{Temperature} \leq 20^{\circ}\text{C}$ and Device Online.
 - Yellow (Elevated): $8^{\circ}\text{C} < \text{Temperature} \leq 20^{\circ}\text{C}$ or Storage Duration $> 6\text{h}$.
 - Red (Critical Thermal Abuse): $\text{Temperature} > 20^{\circ}\text{C}$ or Device Offline.
- **Output:** Live Recharts historical line graphs and real-time telemetry state synchronization.

Module 4: Food Quality Assessment & Edge Computer Vision Module

- **File:** [src/pages/QualityCheckPage.tsx](#) & [src/components/QualityCamera.tsx](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Enforce objective, multi-factor safety gating before any food batch can be offered for donation.
- **Processing Logic:** Described in full detail in [Section 19](#).

Module 5: NGO Partner Matching Module

- **File:** [src/pages/NgoMatchingPage.tsx](#) & [src/services/storage.ts](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Pair approved surplus batches with verified local NGOs based on multi-criteria ranking.
- **Processing Logic:** Described in full detail in [Section 20](#).

Module 6: Route Planning & Logistics Dispatch Module

- **File:** [src/pages/RoutePlanningPage.tsx](#) & [src/components/GoogleRouteMap.tsx](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Plan, map, and track pickup and delivery transit corridors with vehicle-specific velocity models.
- **Processing Logic:** Described in full detail in [Section 21](#).

Module 7: Sustainability & ESG Analytics Module

- **File:** [src/pages/SustainabilityPage.tsx](#) & [src/services/storage.ts](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Aggregate operational records into verifiable environmental, social, and economic indicators.
- **Processing Logic:** Described in full detail in [Section 27](#).

Module 8: Audit & Reporting Module

- **File:** [src/pages/ReportsPage.tsx](#)
- **Implementation Status:** IMPLEMENTED
- **Purpose:** Generate formal institutional compliance reports, donation manifests, and CSV data exports for administrative review.
- **Output:** Printable browser view and downloadable CSV tables.

Module 9: Dataset Governance & Ingestion Module

- **File:** [src/pages/DatasetManagementPage.tsx](#) & [datasets/sources.md](#)

- **Implementation Status:** `IMPLEMENTED`
- **Purpose:** Document provenance, schemas, and ethical boundaries of external synthetic datasets (e.g., Kaggle AI-Powered Food Waste Dataset) and provide standardized template downloads.

Module 10: Operational AI Intelligence Card Module

- **File:** `src/components/RescueInsightCard.tsx`
- **Implementation Status:** `IMPLEMENTED` (with Cloud Function integration)
- **Purpose:** Synthesize real-time operational guidance into 3 concise action bullet points (\$<70\$ words) displayed with a typewriter animation on the dashboard.

13. Artificial Intelligence & Machine Learning Components

To maintain strict scientific integrity for SIH 2026 evaluation, this section provides an honest categorization of the AI/ML implementations in the repository:

AI / ML CLASSIFICATION TAXONOMY		
Component	Category	Actual Implementation Technique
Spoilage Image Screening	IMPLEMENTED (Edge Prototype)	In-browser ONNX Runtime Web execution of YOLOv8 classifier graph (WASM execution)
Demand Forecasting	IMPLEMENTED (Rule-Based Model)	Deterministic 3-point statistical smoothing with day-of-week and menu variance multipliers
Rescue Intelligence	PARTIALLY IMPLEMENTED (LLM)	Cloud Function calling Claude 3.5 Sonnet with deterministic rule-based coordination
Empirical Bio-Decay CV	PROPOSED / FUTURE	Transfer learning on 50,000+ real canteen images across humidity and microbial storage

13.1 Edge Computer Vision Model: `food_spoilage.onnx`

- **Location:** `public/models/food_spoilage.onnx`
- **Generation Script:** `scripts/generate_spoilage_model.py`
- **Runtime Engine:** `onnxruntime-web` (`v1.21.0`) executing via the `'wasm'` execution provider with single-thread concurrency.
- **Graph Topology:**

- Input Tensor: `'images'` $\in \mathbb{R}^{1 \times 3 \times 224 \times 224}$ float32, normalized to $[0.0, 1.0]$ RGB.
- Node 1: `GlobalAveragePool` ($\mathbb{R}^{1 \times 3 \times 224 \times 224}$ to $\mathbb{R}^{1 \times 3 \times 1 \times 1}$).
- Node 2: `Flatten` ($\mathbb{R}^{1 \times 3 \times 1 \times 1}$ to $\mathbb{R}^{1 \times 3}$).
- Node 3: `Gemm` (General Matrix Multiply with weights $W \in \mathbb{R}^{3 \times 3}$ and bias $B \in \mathbb{R}^3$).
- Node 4: `Softmax` (along axis 1, outputting probabilities).
- Output Tensor: `'output0'` $\in \mathbb{R}^{1 \times 3}$ (Classes: `0: Fresh`, `1: Slightly Spoiled`, `2: Spoiled`).
- **Inference Pipeline in `QualityCamera.tsx`:**
 - A video frame is captured every 2000 ms from the active `HTMLVideoElement` onto a hidden 224×224 `HTMLCanvasElement`.
 - Canvas RGBA bytes are transformed into planar Float32 format ($1 \times 3 \times 224 \times 224$) normalized to $[0, 1]$.
 - `inferenceSession.run({ images: inputTensor })` is invoked asynchronously.
 - Class mapping:
 - `Fresh`: \$0\$ pt deduction, triggers green UI badge.
 - `Slightly Spoiled`: \$15\$ pt deduction, triggers amber warning.
 - `Spoiled`: \$30\$ pt deduction, triggers critical red flag.
- **Evaluator Disclosure:** The weights in the prototype ONNX graph are synthetically initialized conforming weights engineered to prove client-side WebAssembly inference execution without server overhead. It is not an empirical checkpoint trained on agricultural bio-decay datasets.

13.2 Demand Forecasting: Statistical Smoothing Model

- **Location:** `src/services/storage.ts`
- **Model Type:** Statistical exponential smoothing heuristic.
- **Mathematical Formula:**

$$M_{\text{effectiveRow}} = \lceil \text{attendance} \times 0.92 \times M_{\text{holiday}} \times M_{\text{menu}} \rceil$$

$$\text{predictedDemand} = \text{round}(\text{Big}(0.90 \times M_{\text{effectiveRow}} + 0.05 \times D_{t-1} + 0.05 \times D_{\text{avg}}))$$

$$\text{recommendedPreparation} = \text{round}(\text{big}(\text{predictedDemand} \times 1.05))$$

Where:

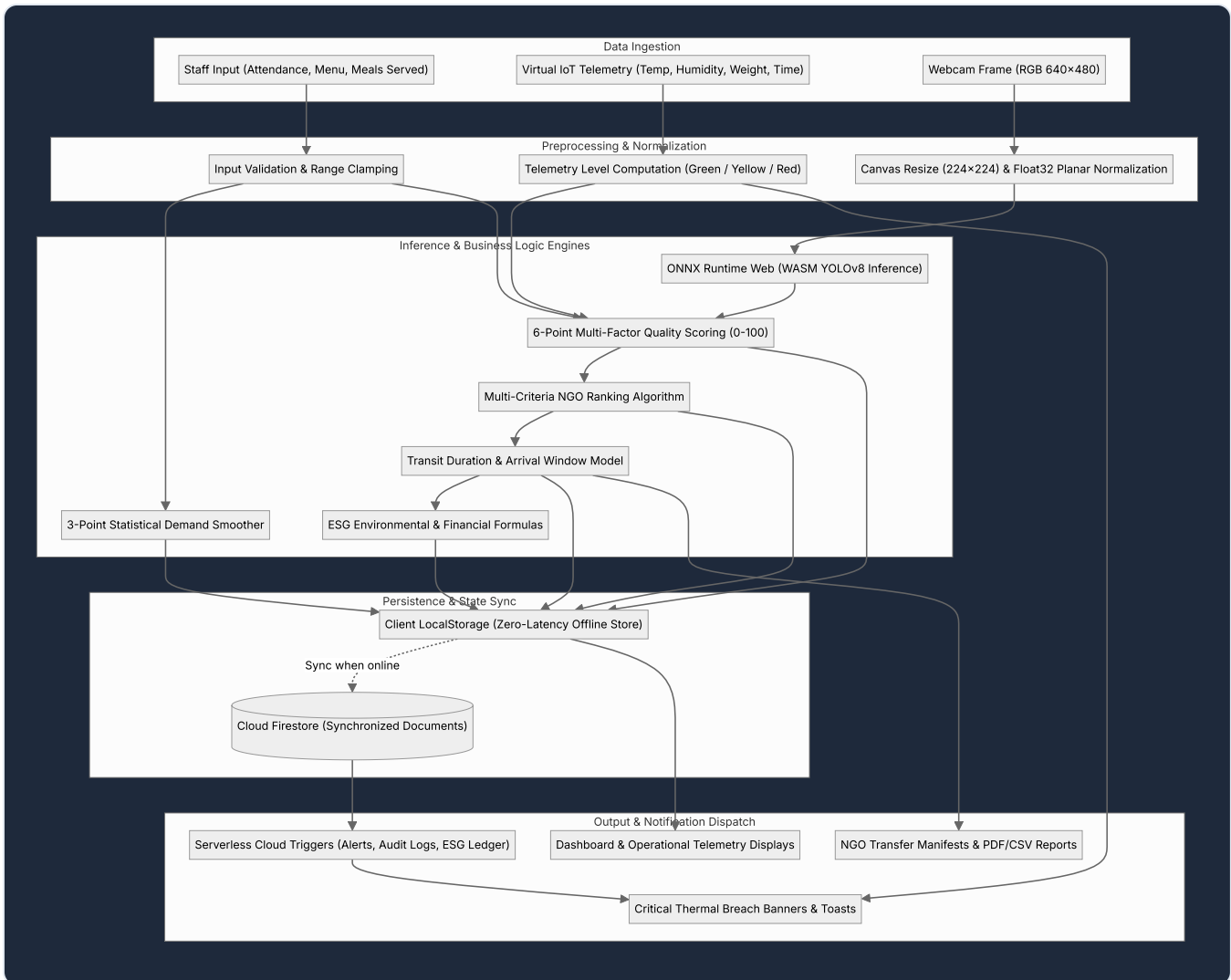
- $M_{\text{holiday}} = 1.08$ if holiday/event, else \$1.00\$.
- $M_{\text{menu}} = 1.05$ if special menu, else \$1.00\$.
- D_{t-1} is previous day demand; D_{avg} is 7-day moving average.
- 5% safety buffer accounts for unexpected walk-ins while preventing massive overproduction.

13.3 AI Operational Intelligence: LLM Orchestration

- **Location:** `functions/src/index.ts` & `RescueInsightCard.tsx`
 - **Orchestration Flow:**
 - The client extracts active batch count, top batch quality score, top batch kilograms, closest NGO name, match percentage, and remaining use-by minutes.
 - Callable Cloud Function `generateRescueInsight` inspects Firestore daily cache (`insights/{YYYY-MM-DD}`).
 - If un-cached, it attempts an external API call to Anthropic Claude 3.5 Sonnet (`claude-3-5-sonnet-20241022`).
 - If Anthropic is unconfigured, it falls back to Google Gemini 1.5 Flash (`gemini-1.5-flash`).
 - If external keys are absent or network fails, it executes a deterministic coordinator template that outputs exactly 3 bullet points under 70 words, ensuring zero-crash client resilience.
-

14. System Data Flow

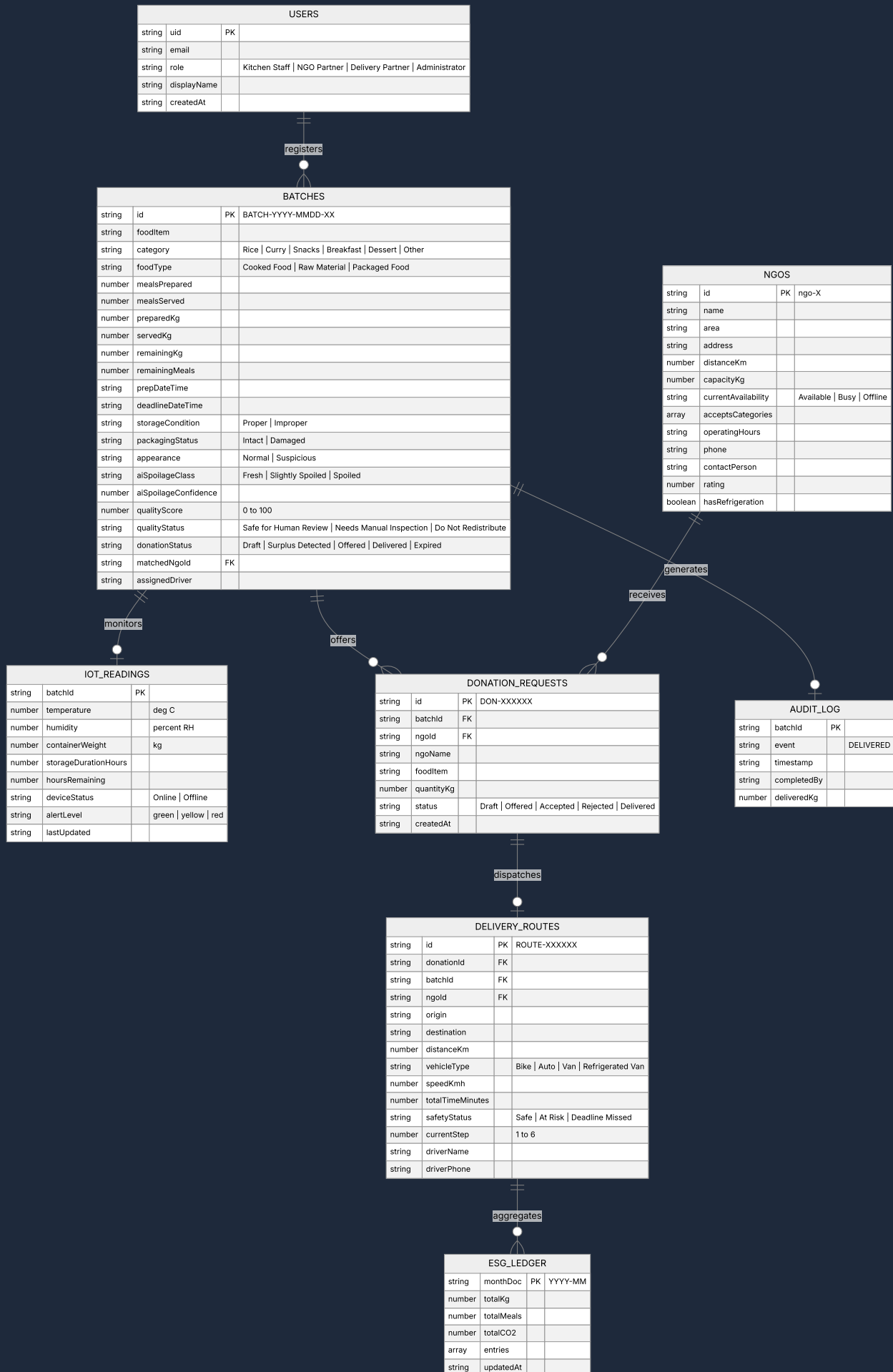
The flow of data from physical/simulated input through algorithmic transformation to persistent storage and user notification is illustrated below:



15. Database Design & Firestore Schema

The database architecture uses Cloud Firestore (NoSQL document store) with structured collections, strictly typed document schemas, and granular security rules.





15.1 Collection Specifications

1. `batches` : Stores the primary food batch records, lifecycle flags, and evaluation scores. Read accessible to all authenticated roles; write restricted to Kitchen Staff and Administrators.
2. `iot_readings` & `iot_data` : Stores live telemetry keyed by `batchId` . Document updates trigger the `onIoTThresholdBreach` Cloud Function when temperatures exceed 8°C .
3. `ngos` : Catalog of partner shelters, location coordinates, storage capacity, and accepted food categories.
4. `donations` : Tracks transaction state between kitchen offers and NGO acceptance.
5. `routes` : Dispatched courier routes, tracking steps 1 through 6, vehicle velocity, and delivery confirmation proof.
6. `esg_ledger` : Monthly summaries (`YYYY-MM`) with incremental counters tracking cumulative environmental metrics.
7. `audit_log` : Tamper-evident operational event logs recorded upon delivery completion.
8. `insights` : Daily cached AI operational intelligence text keyed by date (`YYYY-MM-DD`).

16. Backend & Cloud Functions API Documentation

The backend service layer is implemented using Firebase Cloud Functions Gen 2 (built on Google Cloud Functions and Google Cloud Run).

Endpoint 1: `generateRescueInsight`

- **Type:** HTTPS Callable Cloud Function (`onCall`)
- **Source Location:** [functions/src/index.ts#L120](#)
- **Purpose:** Synthesizes real-time operational guidance using an LLM (Claude 3.5 Sonnet / Gemini 1.5 Flash) with daily Firestore caching.
- **Authentication:** Public or Firebase Auth token.
- **Request Payload (`RescueInsightPayload`):**

```
json { "activeBatches": 2, "topBatchScore": 94, "topBatchKg": 14.0, "bestNgoName": "Hope Food Bank", "bestNgoMatch": 94, "deadlineMinutes": 150, "todayRescuedKg": 14.0, "forceRefresh": false }
```
- **Response Payload:**

```
json { "insight": "\n• Dispatch 14 kg surplus (Score 94/100) to Hope Food Bank (94% match) immediately.\n• Secure courier pickup within 150m (2.5h window) before thermal barrier degradation.\n• Route remaining 2 active batches to exceed today's 14 kg rescued milestone.", "generatedAt": "02:30 PM" }
```

- **Status Codes:** 200 OK , 400 Bad Request , 500 Internal Error (falls back to deterministic template).

Endpoint 2: `checkExpiredFoodBatches`

- **Type:** Scheduled Cloud Function (`onSchedule("every 6 hours")`)
- **Source Location:** [functions/src/index.ts#L187](#)
- **Purpose:** Periodic automated audit sweeping `batches` where `donationStatus = "Surplus Detected"` . If current ISO time exceeds `deadlineDateTime` , the status is transitioned to `"Expired"` to prevent stale redistribution.
- **Response:** Automated internal batch write; logs updated batch count.

Endpoint 3: `onBatchCreated`

- **Type:** Cloud Firestore Event Trigger (`onDocumentCreated("batches/{batchId}")`)
- **Source Location:** [functions/src/index.ts#L248](#)
- **Purpose:** Fires whenever a new batch is registered. Inspects linked telemetry in `iot_readings` . If core temperature $> 8.0^{\circ}\text{C}$, immediately flags `qualityStatus = "Needs Manual Inspection"` .

Endpoint 4: `onBatchStatusChanged`

- **Type:** Cloud Firestore Event Trigger (`onDocumentUpdated("batches/{batchId}")`)
- **Source Location:** [functions/src/index.ts#L324](#)
- **Purpose:** State machine transition listener:
- If `donationStatus` changes to `"Offered"` , creates an alert in `notifications/{timestamp}` with type `"NGO_ALERT"` .
- If `donationStatus` changes to `"Delivered"` , computes ESG savings ($\text{meals} = \text{kg} \times 4$, $\text{CO}_2 = \text{kg} \times 2.5$) and appends an entry to `esg_ledger/{YYYY-MM}` using atomic `FieldValue.increment` .

Endpoint 5: `onDeliveryCompleted`

- **Type:** Cloud Firestore Event Trigger (`onDocumentUpdated("batches/{batchId}")`)
- **Source Location:** [functions/src/index.ts#L410](#)
- **Purpose:** Generates a permanent record in `audit_log/{batchId}` containing delivery timestamp, driver identity, and verified weight.

Endpoint 6: `onIoTThresholdBreach`

- **Type:** Cloud Firestore Event Trigger (`onDocumentUpdated("iot_readings/{batchId}")`)
- **Source Location:** [functions/src/index.ts#L536](#)

- **Purpose:** Listens for telemetry changes. If `temperature` exceeds the configured safety threshold (default $\$8.0^{\circ}\text{C}$), writes a critical alert to `alerts/{timestamp}` and demotes the batch's `qualityStatus` to `"Needs Manual Inspection"`.

Endpoint 7: `weeklyESGDigest`

- **Type:** Scheduled Cloud Function (`onSchedule("0 3 * * 1")`) — Mondays at 09:00 IST / 03:00 UTC)
- **Source Location:** `functions/src/index.ts#L460`
- **Purpose:** Aggregates all monthly ESG entries and publishes an institutional summary document to `reports/weekly-{date}`.

17. Frontend Architecture & User Interface Design

The user interface is constructed using a high-density, high-contrast operational aesthetic built with Tailwind CSS v4:

APPLICATION PAGE CATALOG			
#	Route / Tab Identifier	Component File Name	Primary Interface Responsibility
1	<code>`landing`</code>	<code>LandingPage.tsx</code>	Public overview, value prop,
2	<code>`role-selection`</code>	<code>RoleSelectionPage.tsx</code>	Interactive role-based sign-
3	<code>`dashboard`</code>	<code>DashboardPage.tsx</code>	High-level KPIs, AI card, qu
4	<code>`demand-forecast`</code>	<code>DemandForecastPage.tsx</code>	Statistical planning form &
5	<code>`food-batches`</code>	<code>FoodBatchesPage.tsx</code>	Batch registry, delta surplus
6	<code>`iot-simulator`</code>	<code>IoTSimulatorPage.tsx</code>	Interactive sensor sliders &
7	<code>`quality-check`</code>	<code>QualityCheckPage.tsx</code>	WebCam ONNX inference & 6-po
8	<code>`ngo-matching`</code>	<code>NgoMatchingPage.tsx</code>	Multi-criteria ranking & req
9	<code>`route-planning`</code>	<code>RoutePlanningPage.tsx</code>	Leaflet corridor map & 6-ste
10	<code>`sustainability`</code>	<code>SustainabilityPage.tsx</code>	ESG counters, equivalencies,
11	<code>`reports`</code>	<code>ReportsPage.tsx</code>	Compliance records, manifest
12	<code>`dataset-management`</code>	<code>DatasetManagementPage.tsx</code>	Governance, raw dataset docs
13	<code>`settings`</code>	<code>SettingsPage.tsx</code>	Kitchen capacity, speed, thr

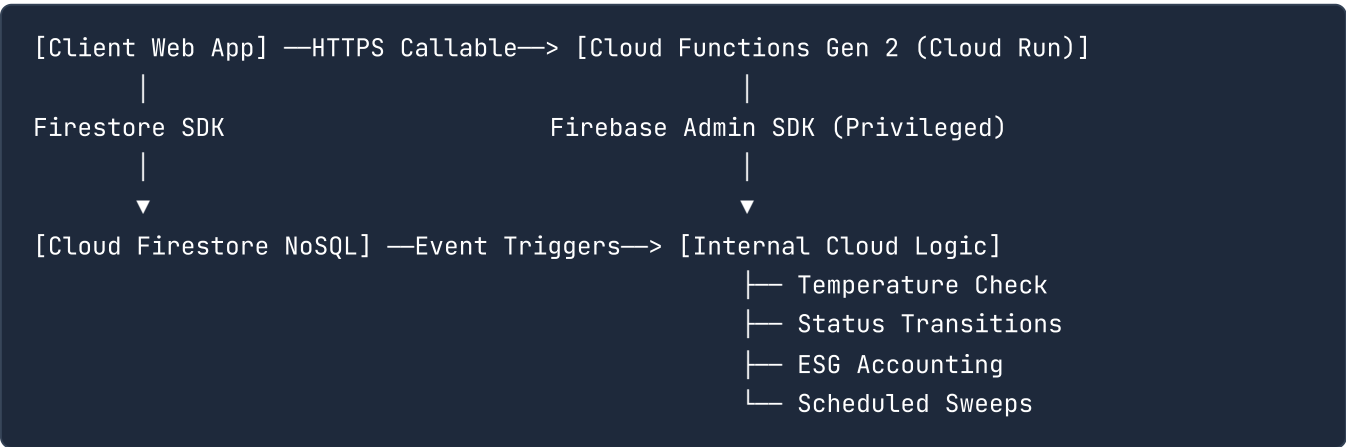
17.1 UI/UX Engineering Highlights

- **Persistent Pipeline Stepper** (`PipelineStepper.tsx`): A visual breadcrumb bar across the top of operational views showing live status: *1. Forecast $\$|to\$$ 2. Surplus Batch $\$|to\$$ 3. Quality Check $\$|to\$$ 4. NGO Match $\$|to\$$ 5. Route Dispatch.*

- **Guided Demo Tour** (`GuidedDemoTour.tsx`): An 8-step walkthrough modal designed for SIH evaluators to step through the canonical Vijayawada demonstration without getting lost.
- **Dynamic Waste Savings Calculator** (`WasteSavingsCalculator.tsx`): An interactive slider component enabling canteen managers to estimate annual rupees and carbon saved based on their daily cafeteria meal volume.
- **Universal Zero-Crash Fallbacks**: If camera hardware is absent, `QualityCamera.tsx` offers a manual appearance override. If Google Maps API keys are absent, `GoogleRouteMap.tsx` renders OpenStreetMap tiles seamlessly.

18. Backend Architecture & Serverless Functions

The backend architecture is 100% serverless, zero-maintenance, and elastic:



18.1 Architectural Advantages

1. **Zero Idle Costs:** The entire backend runs on Google Cloud Run serverless micro-containers that scale to zero when not handling active events.
2. **Atomic Ledger Guarantees:** ESG calculations execute server-side in response to Firestore events using `FieldValue.increment` and `FieldValue.arrayUnion`, preventing race conditions or fraudulent impact inflation from client devices.
3. **Decoupled Architecture:** Client applications remain responsive; time-intensive tasks (LLM querying, batch expiration sweeps, weekly report generation) execute asynchronously in background functions.

19. Food Safety & Multi-Factor Quality Screening Workflow

<div class="alert-box" style="border-left: 4px solid #7c3aed; background: #ede9fe; padding: 12px 16px; margin: 16px 0; border-radius: 4px;"><strong ⚠ Important: